LAPORAN PRAKTIKUM
KOMPUTASI PARALEL DAN TERDISTRIBUSI



Modul ke- : UTS
Judul Modul : Toko Buku API
dengan FastAPI, aiosqlite,
BackgroundTasks, dan asyncio.Lock

Oleh:

Nama : Hafiz Saklinov
NIM : 231220015
Kelas: KOMPUTASI
PARALEL DAN
TERDISTRIBUSI

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS MUHAMMADIYAH PONTIANAK
2026

1. Tujuan

Tujuan pengerjaan UTS ini adalah:

1. Membangun REST API toko buku dengan FastAPI dan SQLite yang diakses lewat aiosqlite.
2. Memproses transaksi di latar belakang dengan BackgroundTasks tanpa memblokir respons.
3. Membuktikan race condition pada pemotongan stok dan mengatasinya dengan asyncio.Lock.
4. Membandingkan kinerja 1 worker dan 4 worker Uvicorn menggunakan Locust.

2. Hasil dan Pembahasan

2.1 Deskripsi Aplikasi

Tema aplikasi adalah toko buku untuk pengelola toko yang mencatat stok dan penjualan. Resource utama books (id, judul, penulis, kategori, harga, stok) dan resource transaksi transactions (id, book_id, qty, status, created_at). Fitur utama: CRUD buku dengan paginasi, transaksi yang langsung dijawab pending lalu diproses di latar belakang, dan pemotongan stok yang dikunci. Batasan: asyncio.Lock hanya berlaku dalam satu proses, pemrosesan disimulasikan dengan asyncio.sleep(3), dan SQLite hanya mengizinkan satu penulis pada satu waktu.

2.2 Jawaban Soal A1: Concurrency vs. Paralelisme

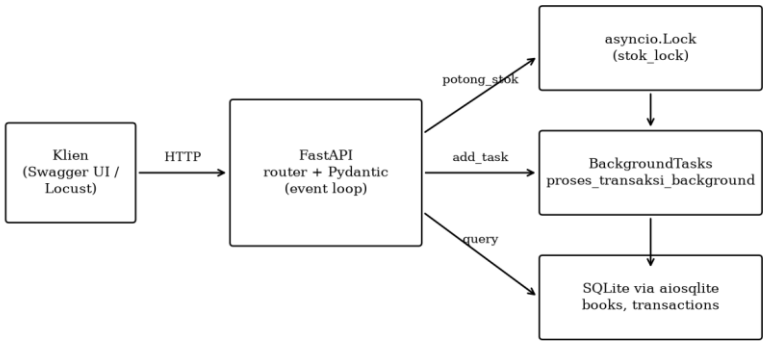
a. Bagian concurrent (I/O-bound). Semua endpoint menunggu I/O: kueri aiosqlite, asyncio.sleep(3) pada proses_transaksi_background, dan jeda 0,01 detik saat pemotongan stok. Selama satu coroutine menunggu di await, event loop melayani request lain, sehingga POST /transactions dapat menjawab pending seketika sementara transaksi lain berjalan di latar belakang.

b. Bagian yang membutuhkan paralelisme (CPU-bound). Hampir tidak ada. Serialisasi JSON dan validasi Pydantic memakai CPU tetapi ringan. Bila kelak ada pekerjaan berat, misalnya membuat laporan penjualan besar, pekerjaan itu memerlukan multiprocessing atau worker terpisah.

c. GIL dan multi-worker. GIL membuat satu proses Python hanya menjalankan satu thread bytecode pada satu waktu, sehingga threading tidak mempercepat kode CPU-bound, dan satu event loop hanya memakai satu inti. Uvicorn dengan --workers N menjalankan N proses

terpisah, masing-masing dengan interpreter, GIL, dan event loop sendiri, sehingga request dapat dikerjakan di beberapa inti sekaligus. Konsekuensinya, memori tidak dibagi: asyncio.Lock di satu worker tidak terlihat oleh worker lain.

2.3 Jawaban Soal A2: Arsitektur dan Alur Data



Gambar 1. Arsitektur aplikasi: FastAPI, asyncio.Lock, BackgroundTasks, dan SQLite (aiosqlite).

Alur satu siklus transaksi: (1) klien mengirim POST /transactions; (2) FastAPI memvalidasi body dengan Pydantic (422 bila salah); (3) router memeriksa buku ada (404 bila tidak); (4) potong_stok masuk async with stok_lock, membaca stok, memeriksa kecukupan, lalu mengurangi stok (409 bila kurang); (5) baris transaksi disimpan dengan status pending dan add_task menjadwalkan pekerja; (6) respons 202 dikirim; (7) setelah respons terkirim, pekerja mengubah status menjadi processing, menunggu asyncio.sleep(3), lalu menyimpan completed atau failed bila terjadi galat; (8) klien memantau lewat GET /transactions/{id}.

2.4 Endpoint

Tabel 1. Daftar endpoint.

Method	Path	Fungsi	Status
GET	/books?skip=&limit=	Daftar buku	200
GET	/books/{id}	Detail buku	200 / 404
POST	/books	Tambah buku	201 / 422
PUT	/books/{id}	Perbarui buku	200 / 404 / 422
DELETE	/books/{id}	Hapus permanen	200 / 404
POST	/transactions	Buat transaksi (pending)	202 / 404 / 409 / 422

Method	Path	Fungsi	Status
GET	/transactions?status=	Daftar, filter status	200
GET	/transactions/{id}	Detail dan status	200 / 404

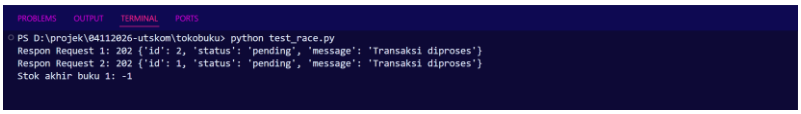
2.5 Pengujian CRUD dan Transaksi



Gambar 2. Luaran demo: CRUD, validasi, dan perubahan status transaksi menjadi completed.

Pada Gambar 2, POST /books mengembalikan 201, PUT mengembalikan data terbaru, buku yang tidak ada menghasilkan 404, dan harga negatif ditolak 422. POST /transactions langsung dijawab 202 pending, lalu status berubah ke processing dan completed setelah sekitar 3 detik (pending hanya bertahan sesaat karena pekerja langsung berjalan). Permintaan qty 99 ditolak 409.

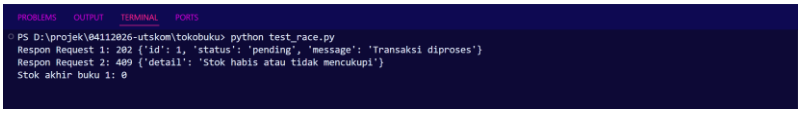
2.6 Race Condition: Tanpa Lock



Gambar 3. Luaran test_race.py tanpa lock: kedua request sukses dan stok akhir -1.

Pada Gambar 3, stok awal 1 tetapi kedua request menerima 202 dan stok akhir menjadi -1. Keduanya membaca stok 1 sebelum salah satunya mengurangi; jeda await asyncio.sleep(0.01) menyerahkan kendali ke event loop tepat di antara baca dan tulis, sehingga request kedua ikut membaca nilai lama. Inilah overselling.

2.7 Race Condition: Dengan Lock



Gambar 4. Luaran test_race.py dengan asyncio.Lock: satu sukses, satu 409, stok akhir 0.

Pada Gambar 4, satu request sukses dan satu ditolak 409 Conflict, dengan stok akhir 0. Blok baca-periksa-kurangi menjadi critical section: request kedua menunggu sampai request pertama selesai, lalu membaca stok 0 dan ditolak. Data tetap konsisten.

2.8 Benchmark 1 Worker dan 4 Worker

Tabel 2. Hasil Locust, tanpa request gagal (total waktu = request / throughput).

Skenario	Worker	Total Waktu	Throughput	Rata-rata Latensi
GET /books	1	1,13 s	88,8 req/s	213 ms
GET /books	4	0,99 s	100,6 req/s	153 ms
POST /transactions	1	2,46 s	20,3 req/s	477 ms
POST /transactions	4	2,02 s	24,8 req/s	383 ms

2.9 Analisis Performa

Empat worker memperbaiki kedua skenario: throughput baca naik sekitar 13% dan latensi turun sekitar 28%; throughput tulis naik sekitar 22% dan latensi turun sekitar 20%. Selisihnya moderat karena sampel kecil (durasi di bawah tiga detik), Locust dan keempat worker berbagi CPU laptop yang sama, dan SQLite hanya mengizinkan satu penulis pada satu waktu. GET /books hanya menjalankan kueri ringan sehingga satu event loop sudah cukup cepat. POST /transactions lebih berat (beberapa koneksi aiosqlite, jeda 0,01 detik di dalam lock, penulisan database), jadi lebih terbantu oleh worker tambahan.

Konsekuensinya bagi keamanan data: asyncio.Lock hanya berlaku dalam satu proses, sehingga dengan 4 worker tiap worker memegang lock sendiri dan stok yang sama masih bisa terjual dua kali. Uji race condition karena itu memakai 1 worker. Untuk proyek akhir, jaminan lintas worker perlu pengunci di tingkat database, misalnya UPDATE atomik dengan syarat stok mencukupi.

3. Kesimpulan

Aplikasi dibangun dengan FastAPI dan aiosqlite; transaksi dijawab pending seketika lalu diselesaikan BackgroundTasks. Tanpa lock, dua request serentak menghasilkan stok -1; dengan asyncio.Lock satu berhasil dan satu ditolak 409. Empat worker menaikkan throughput sekitar 13% (baca) dan 22% (tulis), tetapi lock per proses tidak melindungi stok antar worker.